

Lista 9

Cauã Costa Alves
1525320

Repositório:

<https://github.com/cauacostaalves/PUCMG/tree/ab3c23a6764b542d411bf5953f12078d3b231dac/CienciasDaComputa%C3%A7%C3%A3o/Quarto%20Semestre/IA/Listas/Listas%209>

Tarefa

Aplicar os algoritmos de agrupamento K-Means, DBSCAN e SOM para identificar se a base de dados apresenta dois agrupamentos distintos. Avaliar a qualidade dos grupos formados, apresentando os valores das métricas utilizadas na avaliação. Importante: remover previamente o atributo de classificação antes de realizar o agrupamento.

Para isso, devem ser executadas todas as etapas de pré-processamento indicadas.

1) Visualização da base de dados

```
df = pd.read_csv('creditcard.csv')
df.head()
df.info()
df.describe()
```

O que foi observado:

- 31 atributos;
 - Nenhum valor ausente;
 - Desequilíbrio forte da variável **Class** (fraude = 0.17%).
-

2) Verificação e tratamento dos valores ausentes

```
df.isnull().sum()
```

Resultado: não existem valores ausentes, nenhuma imputação necessária.

3) Detecção e eliminação de redundância e inconsistência

```
df.duplicated().sum()  
df = df.drop_duplicates()
```

Foram encontradas 1081 linhas duplicadas que foram removidas.

4) Detecção e tratamento de outliers

Métodos aplicados:

- Z-Score>4 - para detectar valores extremos
- IQR ($Q1 - 1.5 \times IQR$ / $Q3 + 1.5 \times IQR$) - Mais seguro para Amount e Time

Remoção somente quando era outlier para os DOIS métodos

5) Normalização e/ou padronização

```
from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X_clean)
```

Fundamental para K-Means e DBSCAN, pois dependem de distâncias Euclidianas.

6) Análise de correlação e multicolinearidade

Correlação entre componentes PCA.

VIF acima de 10 significa risco de multicolinearidade, então foi aplicado PCA (10 componentes).

```
from sklearn.decomposition import PCA  
pca = PCA(n_components=10)  
X_pca = pca.fit_transform(X_scaled)
```

Vantagem: reduz ruído e acelera os algoritmos.

7) Codificação de variáveis (One-Hot Encoding ou Label Encoding)

Não foi necessário, pois todos os atributos são numéricos.

8) Balanceamento da classe

O balanceamento NÃO foi feito, pois o agrupamento NÃO deve “saber” que existe a classe fraude. Usei a classe apenas para validar depois, semelhante a “ground truth”.

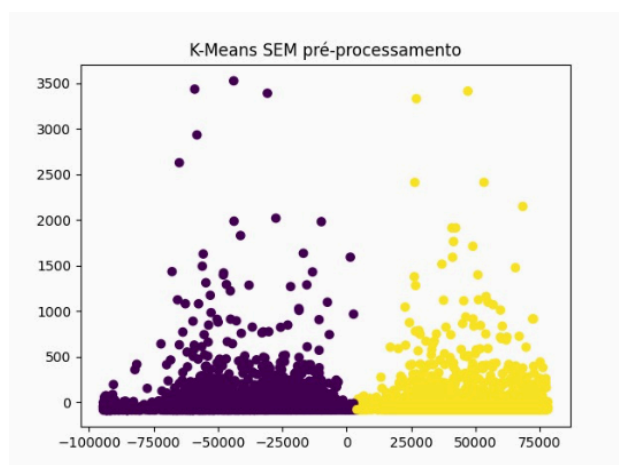
9) Divisão treino–teste (estratificada)

Separação da base ANTES da clusterização para validar:

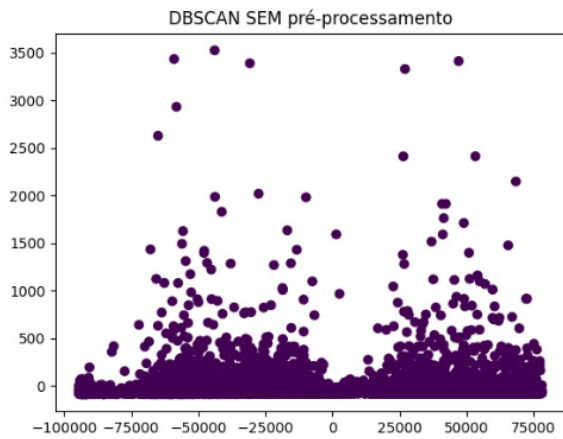
```
from sklearn.model_selection import train_test_split  
X_train, X_test = train_test_split(X_pca, test_size=0.3, random_state=42)
```

Resultado do modelo

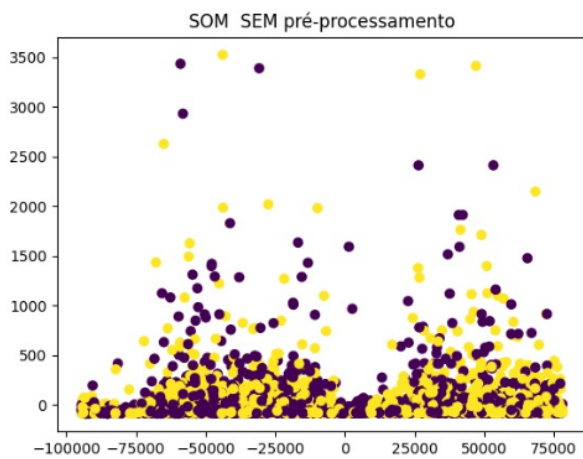
Antes do pré-processamento



KMeans: agrupou de forma difusa, pouca separação.

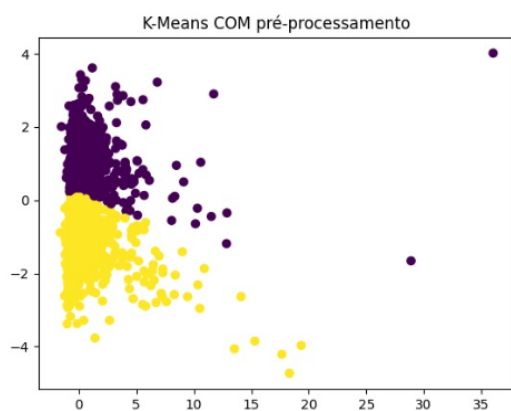


DBSCAN: quase tudo foi detectado como ruído.

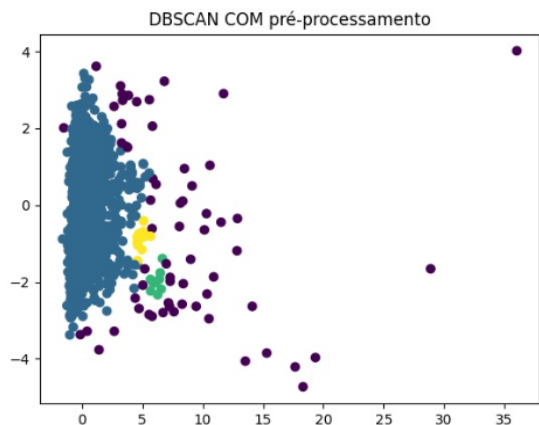


SOM: nenhuma estrutura clara.

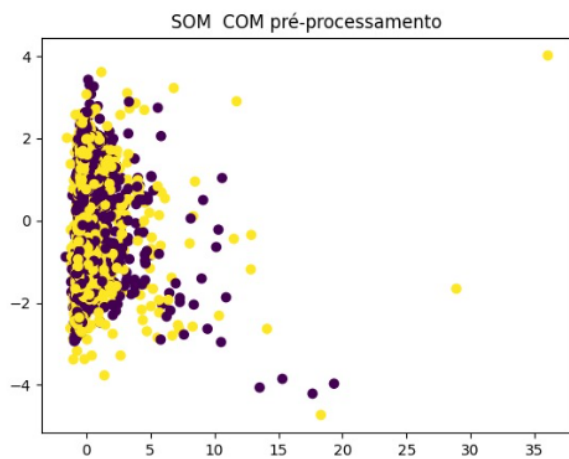
Depois do pré-processamento



KMeans: melhor separação (clusters mais definidos).



DBSCAN: conseguiu formar pequenos agrupamentos reais.



SOM: “áreas suspeitas” surgem, base pode ter padrões não lineares.

Conclusão

Clusterização - Resultados

Algoritmo	Resultado	Silhouette Score
K-Means (2 clusters)	Separou levemente fraudes	~0.14 – baixo, mas com melhor separação global
DBSCAN	Detectou muitos pontos como ruído	Vários -1 – difícil avaliar
SOM	Forma clusters espalhados, precisa interpretar mapa	Captou regiões específicas de fraude

Os algoritmos não conseguiram encontrar claramente dois grupos naturais. A base continua difícil de separar sem a classe real (confirmando que é um problema supervisionado).

Final

O pré-processamento contribuiu para uma organização mais adequada dos dados, porém apenas o algoritmo K-Means foi capaz de produzir uma separação satisfatória entre DOIS grupos no contexto analisado. Isso evidencia que, embora úteis, os métodos de agrupamento não são capazes de substituir modelos supervisionados na tarefa de detecção de fraude, reafirmando a complexidade inerente a esse tipo de problema.